



DuocUC[®] INFORMÁTICA Y
TELECOMUNICACIONES

■ PRUEBAS UNITARIAS EN PROYECTOS DE MICROSERVICIOS

DESARROLLO FULLSTACK I
DSY1103



• Contenidos

- Testing
- Desarrollo de actividades prácticas.

A black and white photograph of a man in a suit standing in a modern office, holding a tablet and smiling. The office has glass partitions and modern furniture. A blue square is in the top right corner.

01

PERSISTENCIA Y ORM SPRING

¿Qué pasa si un proyecto no tiene **Testing**?



• ¿Qué pasa si un proyecto no tiene Testing?

Sin pruebas unitarias, es más probable que el código contenga **errores** y **bugs** que pasan desapercibidos hasta las etapas finales o incluso después del lanzamiento. Las **regresiones** son difíciles de detectar, haciendo que los errores reaparezcan tras cambios en el código. Además, la **refactorización** se vuelve riesgosa, ya que no hay forma de **asegurar que las modificaciones no rompan funcionalidades existentes**. Esto resulta en software menos fiable y aumenta los costos y el tiempo de mantenimiento.



• ¿Qué son las pruebas unitarias?

Las pruebas unitarias son **pruebas automatizadas** que verifican el comportamiento de una pequeña parte del código (una unidad), típicamente una **función** o un **método**, de forma **aislada del resto del sistema**.





• Beneficios

DETECCIÓN TEMPRANA DE ERRORES

Las pruebas unitarias permiten identificar y corregir errores en las etapas iniciales del desarrollo. Esto reduce el costo y el esfuerzo de corregir errores que podrían ser más difíciles de encontrar y arreglar en fases posteriores del desarrollo.

FACILITAN EL REFACTORIZACIÓN Y EL MANTENIMIENTO

Con un conjunto completo de pruebas unitarias, los desarrolladores pueden refactorizar el código con confianza. Las pruebas aseguran que las modificaciones no introduzcan nuevos errores, lo que hace que el mantenimiento del código sea más seguro y eficiente.



• Beneficios

MEJORAN LA CALIDAD DEL CÓDIGO

Las pruebas unitarias fomentan la escritura de código modular y bien estructurado. Al diseñar el código para que sea fácil de probar, se tiende a crear funciones y métodos más pequeños y especializados, lo que mejora la calidad general del código.

DOCUMENTACIÓN ACTIVA

Las pruebas unitarias actúan como una forma de documentación para el código. Cada prueba describe cómo se espera que funcione una unidad específica del código, lo que facilita la comprensión del comportamiento del sistema para otros desarrolladores.



• Beneficios

FACILITAN LA INTEGRACIÓN CONTINUA

Las pruebas unitarias son una parte esencial de las prácticas de integración continua y entrega continua (CI/CD). Las herramientas de CI/CD ejecutan automáticamente las pruebas unitarias con cada cambio de código, lo que asegura una integración y entrega más fluida y confiable.

AHORRO DE TIEMPO Y COSTOS

Aunque la escritura de pruebas unitarias requiere tiempo y esfuerzo inicial, a largo plazo, ahorra tiempo y costos asociados con la depuración y el mantenimiento. La detección temprana de errores y la prevención de regresiones contribuyen a un ciclo de desarrollo más eficiente.

• ¿Qué es JUnit 5?

Es una **plataforma de pruebas** para aplicaciones Java que permite crear y ejecutar **pruebas unitarias**. Ofrece muchas anotaciones y funcionalidades para facilitar la creación de estas pruebas.

JUNIT 5



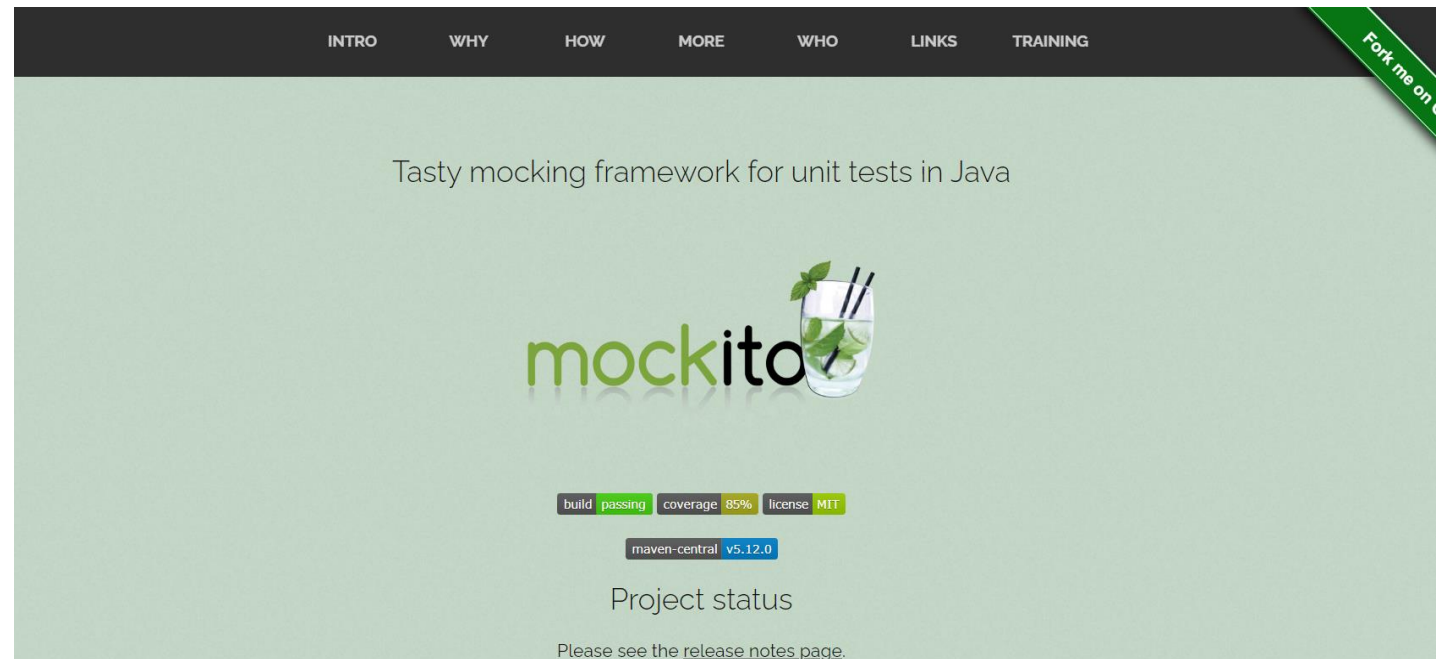
A screenshot of the JUnit 5 website homepage. The page has a clean, modern design with a white background. At the top left is the JUnit 5 logo. In the top right corner, there is a link to 'JUnit 4'. The main heading reads 'The 5th major version of the programmer-friendly testing framework for Java and the JVM'. Below this heading are five buttons: 'User Guide', 'Javadoc', 'Code & Issues', 'Q & A', and 'Support JUnit'. The 'Support JUnit' button is red, while the others are green. Below the buttons, there are two sections. The left section is titled 'JUnit team's statement on the war in Ukraine' and contains a paragraph of text. Below the text is a yellow button that says 'Donate to UN's Ukraine Humanitarian Fund'. The right section is titled 'Latest Release' and shows three links: 'Jupiter v5.11.0-M2', 'Vintage v5.11.0-M2', and 'Platform v1.11.0-M2'. Below these links is a paragraph of text explaining that JUnit artifacts are deployed to Maven Central and can be downloaded using the above links, and that all files are signed using the keys listed in the KEYS file.

• ¿Qué es Mockito?

Mockito es un framework para **pruebas en Java** que permite crear objetos simulados (mocks). Estos mocks permiten simular el comportamiento de dependencias externas, lo cual es útil para aislar la unidad de código que se está probando.



[MOCKITO](#)



• Pruebas Unitarias

Una **prueba unitaria** es una porción de código que verifica el comportamiento de la unidad más pequeña y aislable de tu aplicación, generalmente un método dentro de una clase.

- **Aislamiento:** La clave es el aislamiento. Las pruebas unitarias no deben depender de recursos externos (bases de datos, red, sistemas de archivos).
- **Velocidad:** Deben ejecutarse rápidamente para proporcionar retroalimentación inmediata.

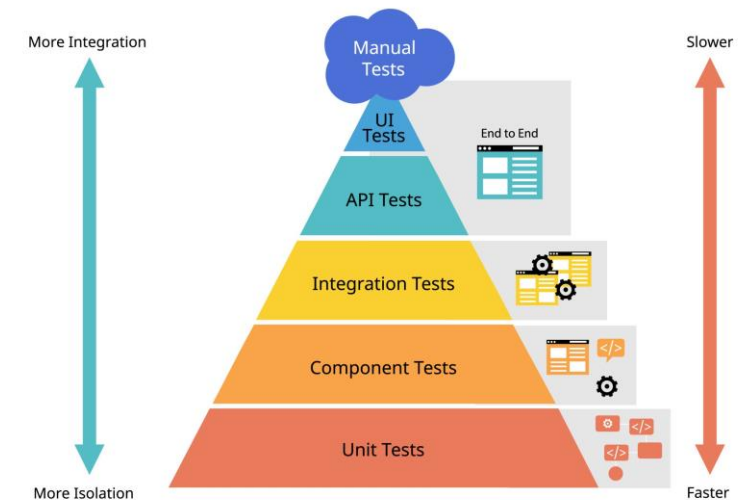
Estructura Given–When–Then (AAA)

Una buena prueba sigue la estructura AAA (Arrange, Act, Assert):

Given (Arrange): Configurar el estado inicial y las dependencias (usando Mockito).

When (Act): Ejecutar el método que se está probando.

Then (Assert): Verificar el resultado usando los métodos Assertions de JUnit.



• Buenas Prácticas de Testing

Práctica	Descripción
Nombres Claros	Usar nombres como <code>shouldCalculateTaxCorrectly</code> o <code>shouldThrowExceptionWhenAmountIsZero</code> .
Atomicidad	Un caso de uso por método <code>@Test</code> . Evitar encadenar lógica de prueba.
Sin BD Real	Usar Mockito o H2 en memoria (solo para integración), nunca la BD de desarrollo/producción.
<code>@DisplayName</code>	Usar esta anotación para descripciones legibles en el reporte.

• Plan de Pruebas Unitarias (Documento Breve / README)

Documenta las reglas críticas y su cobertura.

Ejemplo de Documentación (en tu README o un archivo TESTING_PLAN.md):

```
src > test > TESTING_PLAN.md > ## Pruebas Unitarias y Cobertura de Reglas de Negocio > ### Reflexión y Deuda Técnica
1  ## Pruebas Unitarias y Cobertura de Reglas de Negocio
2
3  ### Reglas de Negocio Críticas del Servicio de Órdenes
4
5  1. **Validación de Stock:** No se puede crear una orden si el stock disponible es menor a la cantidad solicitada.
6  2. **Cálculo de Impuestos:** El impuesto (IVA) debe ser el 19% del subtotal para todos los ítems.
7  3. **Límite de Cantidad:** Un cliente no puede ordenar más de 20 unidades de un mismo producto por orden.
8
9  ### Cobertura Actual de Pruebas Unitarias
10
11 | Regla | Estado | Casos Cubiertos | Checklist |
12 | :--- | :--- | :--- | :--- |
13 | 1. Validación de Stock | **Cubierta** | Stock suficiente (feliz), Stock insuficiente (error). | Caso feliz. Caso de error. |
14 | 2. Cálculo de Impuestos | **Cubierta** | Impuesto calculado correctamente para 1 producto, Impuesto para lista de productos. | Regla crítica tiene test. |
15 | 3. Límite de Cantidad | **Pendiente** | (Solo caso feliz para < 20 unidades). | Falta testear el caso de error. |
16
17 ### Reflexión y Deuda Técnica
18
19 * **Riesgo sin probar:** La regla **Límite de Cantidad** no tiene un test de error (al intentar ordenar 21 unidades). Si se viola, el servicio podría crear órdenes inválidas, lo que representa un riesgo **Medio**.
20 * **Acción Futura:** Agregar test para el caso de error de la Regla 3.
```

• ACTIVIDAD PRÁCTICA



Ingresa al AVA y desarrolla las Guías prácticas - Proyecto Biblioteca Duoc del proyecto de testing para luego implementarlas en tu proyecto.

Investiga la implementación de **testing** en modelos, repositorios, servicios y controladores.



02

**¡MUCHAS
GRACIAS!**

DuocUC[®]

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS
ACREDITADO



DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN
INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.